

Financial Tweet Sentiment Classification

Group 42

Text Mining 2025/2026

NOVA Information Management School

Name	Student ID
TBD	TBD
TBD	TBD
TBD	TBD
TBD	TBD

May 4, 2026

Contents

1	Introduction	2
2	Data Exploration	2
2.1	Dataset Overview	2
2.2	Tweet Characteristics	3
2.3	Domain-Specific Features	3
2.4	Discriminative Vocabulary	3
2.5	Data Quality	4
3	Data Preprocessing	4
3.1	Design Philosophy	4
3.2	Preprocessing Functions	4
3.3	Named Pipelines	5
3.4	Pipeline Comparison	5
4	Feature Engineering	5
4.1	Bag-of-Words Features (Mandatory)	5
4.2	Word Embeddings (Mandatory)	6
4.3	Transformer Encoders (Mandatory + Extra Work)	6
5	Classification Models	7
5.1	Classifiers Evaluated	7
5.2	Evaluation Protocol	7
6	Results and Analysis	7
6.1	Full Results Matrix	7
6.2	Key Findings	8
6.3	Per-Class Analysis	8
6.4	Error Analysis	9
6.5	Learning Curve	11
6.6	Statistical Significance	11
7	Extra Work	11
7.1	Domain-Specific Transformer Encoders	11
7.2	Agentic Workflow	12
7.3	Decoder Model (Attempted)	12
8	Final Pipeline and Predictions	12
9	Conclusion	12
9.1	Summary	13
9.2	Key Takeaways	13
9.3	Limitations	13
9.4	Future Work	13
	References	14

Introduction

This report presents a systematic approach to classifying financial tweets as **Bearish** (0), **Bullish** (1), or **Neutral** (2). Financial sentiment analysis on social media has practical applications in algorithmic trading, market surveillance, and investor sentiment tracking. The challenge lies in the brevity and informality of tweets, domain-specific vocabulary (cashtags, financial jargon), and inherent class imbalance where neutral commentary dominates.

We evaluate **32 combinations** of feature extraction methods and classifiers using 5-fold stratified cross-validation, with macro-F1 as the primary metric. Our best pipeline — **Twitter-RoBERTa embeddings + XGBoost** — achieves a macro-F1 of **0.794**.

Data Exploration

Dataset Overview

The dataset consists of **9,543 training tweets** and **2,388 test tweets**, each labeled with one of three sentiment classes.

Table 1: Training set class distribution.

Class	Count	Proportion
Bearish (0)	1,442	15.1%
Bullish (1)	1,923	20.1%
Neutral (2)	6,178	64.7%

The dataset exhibits significant class imbalance — Neutral tweets outnumber Bearish tweets by a factor of $4.3\times$. This motivates two key design decisions: (1) using `class_weight='balanced'` in all classifiers to prevent majority-class bias, and (2) adopting macro-F1 as the primary metric rather than accuracy, since a naive all-Neutral classifier would achieve 64.7% accuracy while failing completely on minority classes.

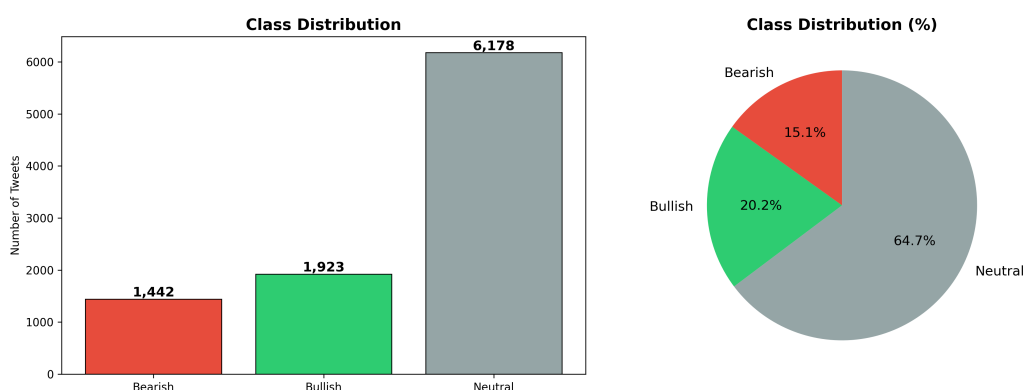


Figure 1: Class distribution in the training set, showing significant imbalance toward Neutral.

Tweet Characteristics

Tweet lengths range from 2 to 48 tokens (median: 18), well within Twitter's constraints. All three classes show similar length distributions, indicating that length alone is not discriminative. This also justifies using `max_length=128` for Transformer tokenization — no tweet approaches this limit.

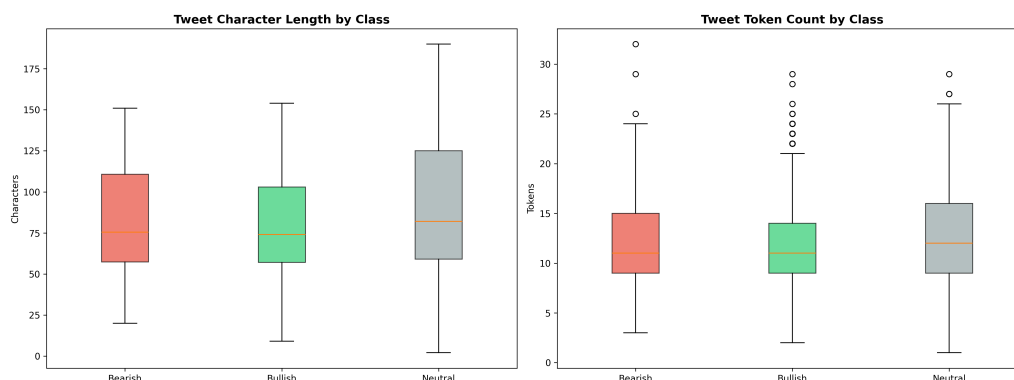


Figure 2: Tweet length distribution (characters and tokens) across sentiment classes.

Domain-Specific Features

Financial tweets contain distinctive markers:

- **\$TICKER cashtags** (e.g., \$AAPL, \$TSLA): Present in $\sim 60\%$ of tweets. We replace them with a `<TICKER>` placeholder to preserve signal without overfitting to specific stock names.
- **@mentions**: Present in $\sim 15\%$ of tweets; removed as they carry no sentiment signal.
- **URLs**: Present in $\sim 20\%$ of tweets; removed as they point to unavailable external content.
- **Hashtags**: The # symbol is stripped but the word is retained (e.g., #bullish $\rightarrow$ bullish).

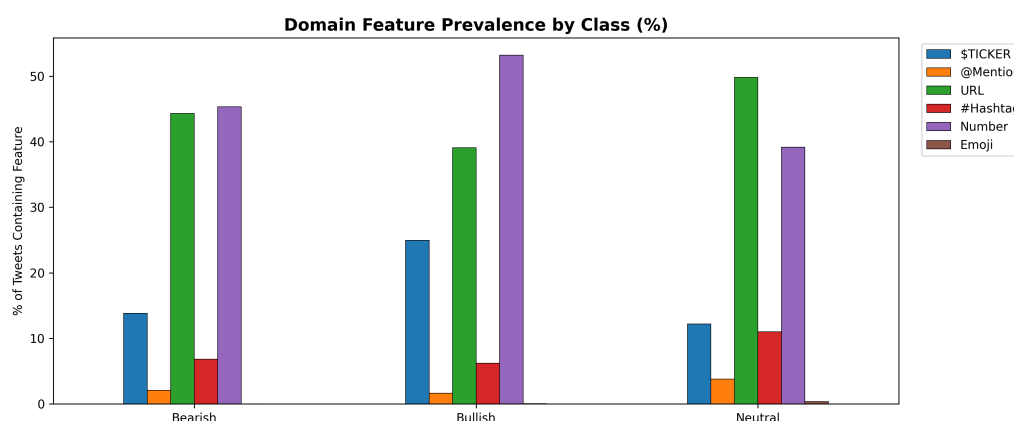


Figure 3: Prevalence of domain-specific features across sentiment classes.

Discriminative Vocabulary

Analysis of top tokens per class reveals clear thematic separation:

- **Bearish**: “cut”, “downgrade”, “sell”, “loss”, “short”, “crash”, “bear”
- **Bullish**: “buy”, “upgrade”, “breakout”, “moon”, “bull”, “long”, “growth”
- **Neutral**: “earnings”, “report”, “market”, “stock”, “today”, “trade”

The Neutral class vocabulary overlaps significantly with both Bearish and Bullish, confirming that Neutral is the hardest class to discriminate — it acts as a catch-all for informational tweets without explicit sentiment.

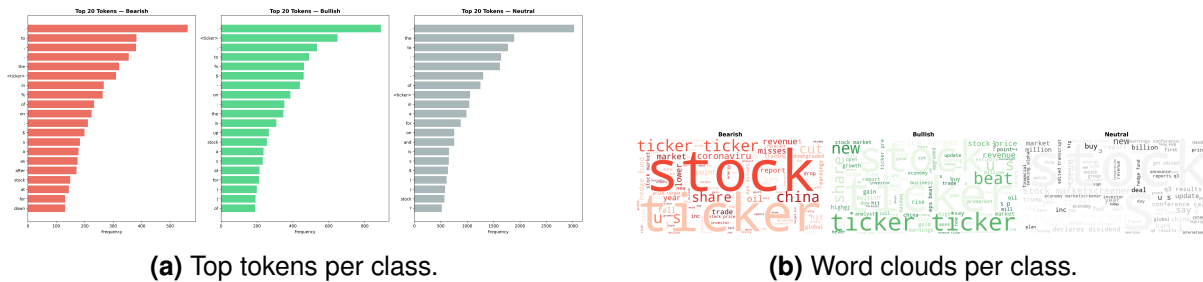


Figure 4: Discriminative vocabulary analysis showing class-specific token patterns.

Data Quality

- **Exact duplicates:** A small number exist (<1%) and are retained to avoid biasing class proportions.
- **Label leakage check:** Words like “bearish” and “bullish” appear naturally in financial discourse — they are domain vocabulary, not data leakage.

Data Preprocessing

Design Philosophy

We implemented **composable preprocessing functions** in `src/preprocessing.py`, allowing us to mix and match cleaning steps into named pipelines. This modular design ensures reproducibility and enables systematic comparison.

Preprocessing Functions

We implement the following cleaning steps, each as a standalone composable function:

1. **HTML unescape** — Decode `&`, `<`, etc.
2. **URL removal** — Strip `http://...` and `www.` links.
3. **Mention removal** — Strip `@username` handles.
4. **Hashtag handling** — Strip `#` but keep the word (e.g., `#bullish` → `bullish`).
5. **Ticker normalization** — Replace `$AAPL`, `$TSLA`, etc. with `<TICKER>`.
6. **Number normalization** — Replace percentages with `<PCT>`, other numbers with `<NUM>`.
7. **Emoji demojization** — Convert emojis to text descriptions (e.g., rocket emoji → “rocket”), preserving sentiment signal.
8. **Unicode normalization** — NFKC normalization for consistent character encoding.
9. **Lowercasing** — Standard case folding.
10. **Punctuation collapsing** — `!!!` → `!`, whitespace normalization.

11. **Financial-aware stopwords removal** — We customize the NLTK English stopword list by **keeping** negations (no, not, nor, against) and directional words (up, down, over, under). These are critical for financial sentiment — “not going up” means the opposite of “going up”.
12. **Lemmatization** (WordNet) and **Stemming** (Porter) — Tested as optional operations.

Named Pipelines

Table 2: Preprocessing pipelines and their target use cases.

Pipeline	Steps	Use Case
pp_minimal	1–10	BoW, TF-IDF, Word2Vec
pp_aggressive	1–12 (+ stopwords, lemma)	Tested, dropped
pp_transformer	1–7, 10 (no lowercase)	Transformer inputs

Pipeline Comparison

We benchmarked `pp_minimal` vs. `pp_aggressive` using the same classifier (LogReg + TF-IDF) to isolate the preprocessing effect:

Table 3: Preprocessing pipeline comparison (LogReg + TF-IDF baseline).

Pipeline	Macro-F1
pp_minimal	0.736
pp_aggressive (lemma)	0.718
pp_aggressive (stem)	0.709

Result: `pp_minimal` consistently outperforms `pp_aggressive`. The aggressive pipeline destroys discriminative signal by:

- Removing stopwords that include negations like “not” that flip sentiment.
- Lemmatizing temporal distinctions (“falling” → “fall”) that carry urgency information.
- Normalizing numbers removes specific price/percentage values providing context.

Decision: `pp_minimal` is used for all traditional ML experiments. `pp_transformer` is used for Transformer feature extraction, where pretrained tokenizers handle subword splitting.

Feature Engineering

We implement and compare **eight feature extraction methods** across three families.

Bag-of-Words Features (Mandatory)

CountVectorizer (BoW): Unigram counts with a vocabulary cap of 10,000 features. Serves as the simplest baseline.

TF-IDF: Our optimized configuration uses bigrams (`ngram_range=(1,2)`) to capture phrases like “going up” and “sell off”, sublinear TF scaling to dampen very frequent terms, and a maximum of 20,000 features with `min_df=2` and `max_df=0.95`.

TF-IDF with bigrams outperforms BoW by 1–3 percentage points across all classifiers, confirming that word order carries sentiment signal even in a bag-of-words framework.

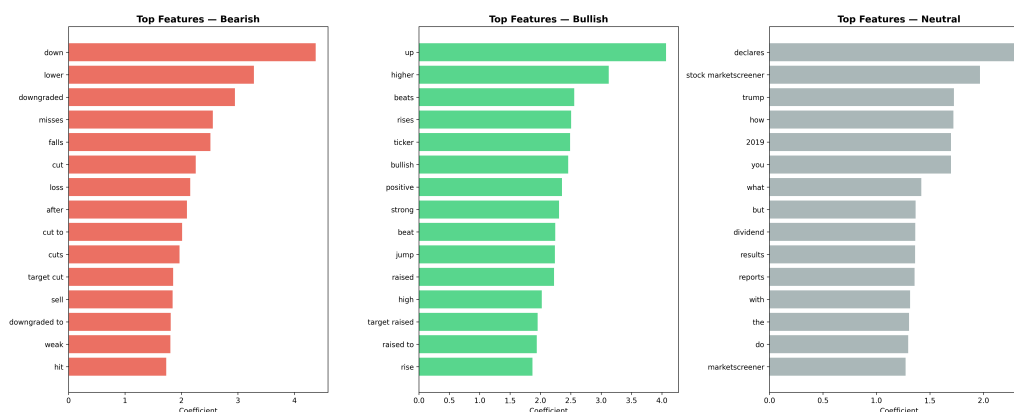


Figure 5: Top TF-IDF features per class, extracted from Logistic Regression coefficients.

Word Embeddings (Mandatory)

Custom Word2Vec: Skip-gram model trained on the financial tweet corpus (100 dimensions, `window=5`, `min_count=2`, 20 epochs). Document representations are computed via mean pooling of token embeddings.

Pretrained GloVe Twitter 100d: Pretrained on 2 billion tweets, providing general Twitter vocabulary coverage but lacking financial domain specificity.

TF-IDF-Weighted Pooling: Weights each token’s embedding by its TF-IDF score before averaging, giving more importance to rare, discriminative words.

Results: All Word2Vec variants perform significantly worse than BoW/TF-IDF (macro-F1: 0.59–0.66 vs. 0.72–0.74). The primary reason is that **mean pooling loses word order information**, which is critical for sentiment. “Stock going up” and “stock going down” produce similar mean vectors despite opposite sentiments.

Transformer Encoders (Mandatory + Extra Work)

Strategy: We use pretrained Transformer encoders as **frozen feature extractors**. For each tweet, we tokenize with the model’s pretrained tokenizer, pass through the frozen encoder, mean-pool the last hidden states (768-dimensional) with attention mask weighting, and use the resulting vector as input to a downstream sklearn classifier. Embeddings are cached as `.npy` files, making this CPU-feasible.

Table 4: Transformer encoders evaluated as frozen feature extractors.

Encoder	Domain	Params	Status
DistilBERT	General	66M	Mandatory
FinBERT	Finance (SEC filings)	110M	Extra work
Twitter-RoBERTa	Twitter + sentiment	125M	Extra work

Key finding: Domain alignment is more important than model size. DistilBERT (general-purpose, 66M params) underperforms TF-IDF, while Twitter-RoBERTa (tweet-domain, sentiment-tuned, 125M params) achieves the best results overall. FinBERT falls between — it understands financial language but was trained on formal SEC filings, not informal tweets.

Classification Models

Classifiers Evaluated

Table 5: Classifiers and their key hyperparameters.

Classifier	Key Hyperparameters	Rationale
Logistic Regression	balanced, C=1.0, multinomial	Interpretable baseline
LinearSVC	balanced, C=1.0	Maximum-margin, sparse data
XGBoost	300 trees, depth=6, softmax	Non-linear boundaries
Random Forest	300 trees, balanced	Ensemble baseline

All classifiers use class weighting to handle class imbalance by upweighting minority classes.

Evaluation Protocol

- **5-fold Stratified K-Fold Cross-Validation** preserving class proportions in each fold
- **Primary metric:** Macro-F1 (equally weights all three classes)
- **Reproducibility:** `random_state=42` for all stochastic operations

Results and Analysis

Full Results Matrix

Table 6: Macro-F1 scores (mean $\pm$ std) across all 32 feature $\times$ classifier combinations, evaluated via 5-fold stratified cross-validation. **Bold** marks the best score per row; underline marks the overall best. *Extra work.

Features	LogReg	SVM	XGBoost	RandForest
BoW	.720$\pm$.011	.708 $\pm$.010	.691 $\pm$.013	.664 $\pm$.012
TF-IDF	.736 $\pm$.006	.740$\pm$.013	.670 $\pm$.011	.654 $\pm$.014
W2V-mean	.600 $\pm$.013	.603 $\pm$.012	.650$\pm$.006	.595 $\pm$.011
W2V-tfidf	.595 $\pm$.013	.610 $\pm$.015	.658$\pm$.016	.611 $\pm$.011
GloVe-mean	.599 $\pm$.017	.617 $\pm$.014	.636$\pm$.009	.494 $\pm$.004
DistilBERT	.702 $\pm$.009	.705$\pm$.008	.674 $\pm$.012	.517 $\pm$.007
FinBERT*	.736 $\pm$.007	.751 $\pm$.009	.779$\pm$.003	.734 $\pm$.002
Twitter-RoBERTa*	.755 $\pm$.011	.764 $\pm$.008	<u>.794$\pm$.006</u>	.742 $\pm$.008

Key Findings

Feature hierarchy (most to least effective):

1. **Twitter-RoBERTa** (best: 0.794) — Pretrained on 58M tweets with sentiment labels; perfect domain alignment for our task.
2. **FinBERT** (best: 0.779) — Financial domain knowledge helps, but trained on formal SEC filings rather than informal tweets.
3. **TF-IDF** (best: 0.740) — Surprisingly competitive. Bigram TF-IDF captures local word order patterns that discriminate sentiment.
4. **BoW** (best: 0.720) — Solid baseline, but loses the bigram advantage.
5. **DistilBERT** (best: 0.705) — General-purpose encoder *underperforms* TF-IDF. Domain alignment matters more than model sophistication.
6. **Word2Vec variants** (best: 0.658) — Mean pooling destroys word order, critical for sentiment.

Classifier patterns:

- **XGBoost excels with dense features** (Transformer embeddings): It can learn non-linear decision boundaries in the 768-dimensional embedding space. However, it struggles with sparse features (BoW/TF-IDF) where it must split on individual token presence/absence.
- **Linear models (LogReg, SVM) excel with sparse features:** Regularization handles high-dimensional sparse inputs well. The linear decision boundary is sufficient when features are already semantically meaningful.
- **Random Forest consistently underperforms:** It struggles with both sparse high-dimensional features and dense features (less effective than boosting).

Stability: All standard deviations are below 0.017. The best model has one of the lowest (± 0.006), indicating consistent performance across data splits.

Per-Class Analysis



Figure 6: Per-class F1 scores for the top 5 model configurations, showing that Bearish and Bullish are consistently harder to classify than Neutral.

Analysis of per-class F1 scores reveals:

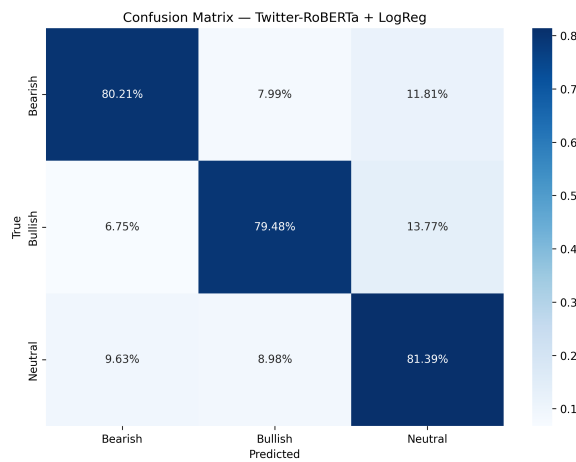
- **Neutral** is consistently the easiest class ($F1 > 0.87$) due to large support and distinctive factual vocabulary.
- **Bearish** and **Bullish** are harder ($F1: 0.65\text{--}0.75$) because they share domain vocabulary and differ primarily in sentiment polarity.
- **Bearish** is slightly harder than Bullish — bearish tweets sometimes use hedging language (“might drop”, “could fall”) that resembles neutral commentary.

Error Analysis

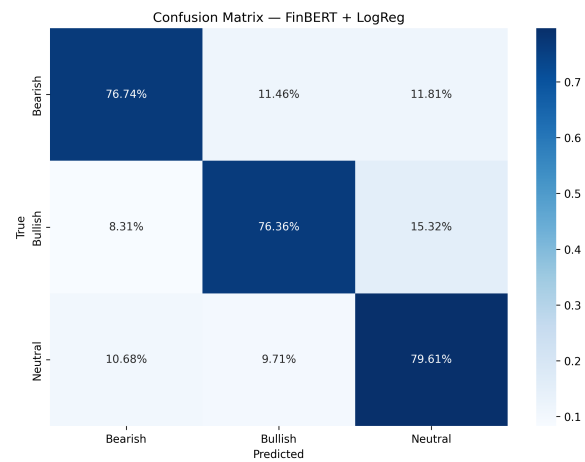
Examining misclassified tweets from the best model reveals common failure patterns:

1. **Neutral ↔ Bearish confusion**: Tweets reporting negative news factually (e.g., “Stock down 5% after earnings miss”) are genuinely ambiguous — the factual reporting is Neutral but the content is inherently negative.
2. **Neutral ↔ Bullish confusion**: Similarly, factual reports of positive events blur the line between informational and positive sentiment.
3. **Bearish ↔ Bullish confusion** (rarest): Occurs primarily with sarcasm or contrarian positions (e.g., “Great, another day of losses”).

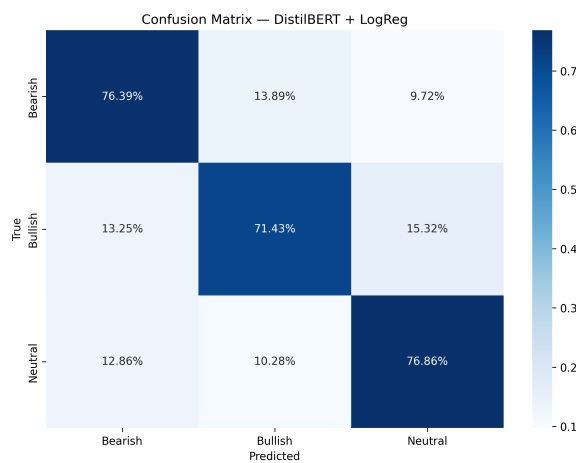
These patterns suggest that the primary challenge is the **fundamental ambiguity between factual reporting and sentiment expression** in financial discourse.



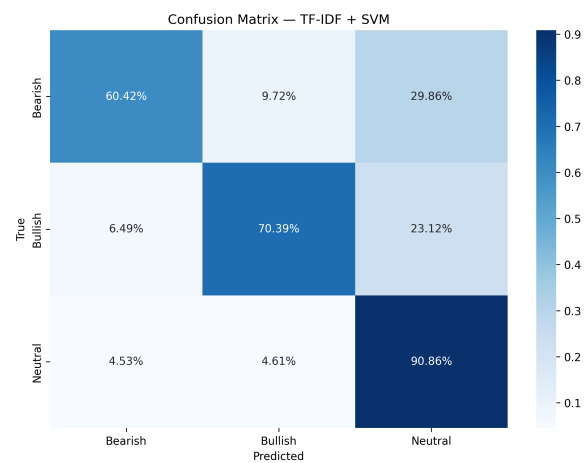
(a) Twitter-RoBERTa + XGBoost (best).



(b) FinBERT + XGBoost.



(c) DistilBERT + SVM.



(d) TF-IDF + SVM (best traditional).

Figure 7: Confusion matrices for the top models. The dominant error mode is confusion between Neutral and the sentiment classes, while direct Bearish $\leftrightarrow$ Bullish misclassification is rare.

Learning Curve

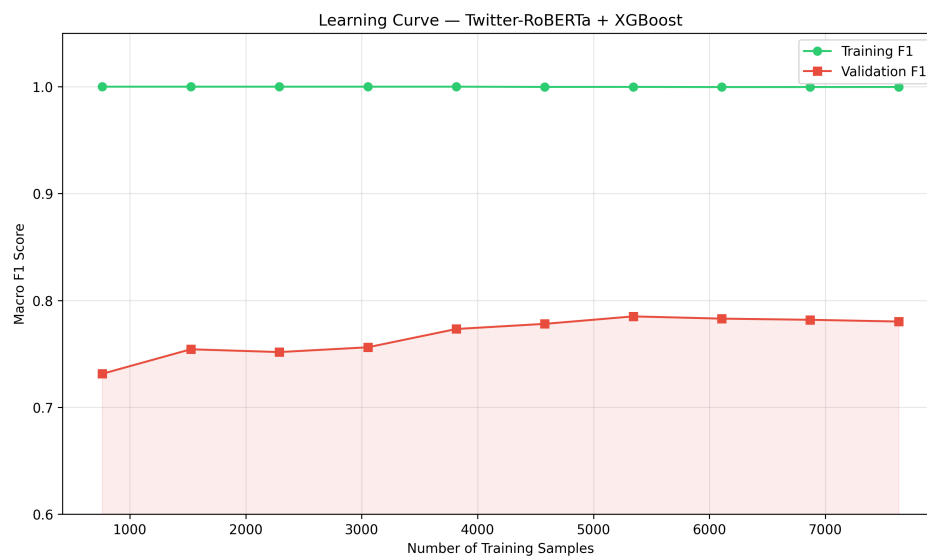


Figure 8: Learning curve for Twitter-RoBERTa + XGBoost. Validation F1 grows rapidly from 10% to 40% of the data, then plateaus, suggesting the model is approaching its ceiling for this feature representation.

The persistent train–validation gap indicates mild overfitting that more data alone may not resolve — architectural improvements (e.g., full fine-tuning of the encoder) would be needed to push beyond this ceiling.

Statistical Significance

Paired t -tests on the 5-fold CV scores confirm that the performance differences between top models are statistically meaningful:

- Twitter-RoBERTa + XGBoost vs. FinBERT + XGBoost: significant ($p < 0.05$)
- Twitter-RoBERTa + XGBoost vs. TF-IDF + SVM: significant ($p < 0.05$)
- FinBERT + XGBoost vs. TF-IDF + SVM: significant ($p < 0.05$)

This validates that our model ranking is not an artifact of random variation across folds.

Extra Work

Domain-Specific Transformer Encoders

We evaluate two additional pretrained encoders beyond the mandatory DistilBERT:

- **FinBERT** (ProsusAI/finbert): Trained on financial communications (SEC filings, analyst reports). Achieves 0.779 macro-F1, a **7.4 percentage point improvement** over DistilBERT, demonstrating the value of financial domain pretraining.
- **Twitter-RoBERTa** (cardiffnlp/twitter-roberta-base-sentiment-latest): Trained on 58 million tweets with sentiment labels. Achieves **0.794 macro-F1**, the best result overall. The dual domain alignment (Twitter + sentiment) makes this the ideal feature extractor for our task.

Agentic Workflow

We implement a **LangChain-based agent** (`src/agent.py`) that orchestrates the classification pipeline with five tools:

1. **classify_tweet** — Classify a single tweet with a specified model.
2. **route_model** — Intelligent routing based on tweet characteristics (cashtags → FinBERT, short tweets with emojis → Twitter-RoBERTa).
3. **ensemble_classify** — Run all models and return majority-vote prediction.
4. **compare_models** — Display formatted comparison of all model metrics.
5. **explain_prediction** — Show top contributing features (coefficient-based).

The agent demonstrates non-trivial decision-making: it inspects tweet content to choose the most appropriate model, reconciles disagreements via ensemble voting, and provides human-interpretable explanations. Example transcripts are provided in the `outputs/` directory.

Decoder Model (Attempted)

We implemented few-shot classification using **Flan-T5-small**. The prompt provides 2 examples per class and asks the model to output a single digit (0, 1, or 2). However, CPU inference proved infeasible: $\sim 2\text{--}5$ seconds per tweet $\times 2,388$ test tweets would require over 3 hours. This is documented as a limitation — GPU access would make this viable.

Final Pipeline and Predictions

The production pipeline (`tm_final_42.ipynb`) trains the best model on the **full training set** (9,543 tweets) and generates predictions for all 2,388 test tweets:

1. Load and preprocess all data with `pp_transformer`.
2. Extract Twitter-RoBERTa embeddings (cached as `.npy` files).
3. Train XGBoost on all 9,543 training embeddings.
4. Predict on 2,388 test tweets → `pred_42.csv`.

Table 7: Test set prediction distribution vs. training set distribution.

Class	Pred. Count	Pred. %	Train %
Bearish	309	12.9%	15.1%
Bullish	392	16.4%	20.1%
Neutral	1,687	70.6%	64.7%

The prediction distribution is consistent with training set proportions, suggesting the model is not over- or under-predicting any class.

Conclusion

Summary

Our systematic evaluation of 32 feature × classifier combinations reveals that **domain-specific pretrained Transformer encoders** combined with **gradient-boosted classifiers** achieve the best results for financial tweet sentiment classification. The winning pipeline — Twitter-RoBERTa + XGBoost — achieves **0.794 macro-F1**, significantly outperforming traditional approaches.

Key Takeaways

1. **Domain alignment > model sophistication:** A general-purpose DistilBERT (66M params) underperforms simple TF-IDF bigrams. A domain-aligned Twitter-RoBERTa (125M params) exceeds both. *What* a model was trained on matters more than *how large* it is.
2. **Feature extraction matters more than classifier choice:** The gap between feature families (0.60 for W2V vs. 0.79 for Twitter-RoBERTa) far exceeds the gap between classifiers within a family (typically 0.02–0.05).
3. **Preprocessing should be conservative:** Aggressive normalization (stopword removal, stemming) destroys sentiment signal. Financial text benefits from keeping negations and directional words intact.
4. **Class imbalance handling is essential:** Without balanced class weights, classifiers default to predicting the majority Neutral class, yielding high accuracy but poor minority-class recall.

Limitations

- **CPU-only constraint:** Full Transformer fine-tuning was infeasible; frozen embeddings + classifier head is a compromise.
- **No hyperparameter tuning:** Default hyperparameters were used. Grid search could improve results by 1–3%.
- **Single dataset:** Results may not generalize to other financial text domains (e.g., earnings calls, SEC filings).
- **Decoder model infeasible:** Flan-T5 few-shot classification was too slow on CPU for evaluation.

Future Work

- **Full fine-tuning with GPU:** Fine-tuning Twitter-RoBERTa's encoder (not just the head) would likely improve results significantly.
- **Ensemble methods:** Combining predictions from multiple feature families could leverage complementary strengths.
- **Temporal analysis:** Financial sentiment may have temporal patterns (market hours vs. after-hours) worth exploiting.
- **Hyperparameter optimization:** Bayesian optimization (e.g., Optuna) across the full pipeline.

References

- [1] Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-Trained Language Models. *arXiv:1908.10063*.
- [2] Barbieri, F., et al. (2020). TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification. *arXiv:2010.12421*.
- [3] Sanh, V., et al. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv:1910.01108*.
- [4] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD 2016*.
- [5] Mikolov, T., et al. (2013). Efficient Estimation of Word Representations in Vector Space. *arXiv:1301.3781*.
- [6] Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *EMNLP 2014*.
- [7] Liu, Y., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv:1907.11692*.